Mise en route Base de Données (Laravel)

- **Configurer une base de données dans Laravel**
- **Comprendre comment Laravel se connecte à MySQL**
- **Exécuter des requêtes SQL (CRUD)**
- **Comprendre et utiliser les transactions**
- **Utiliser les commandes Artisan (CLI)**
- **Comprendre ce qu'est une migration**
- **Comprendre ce qu'est un seeder**

PARTIE 1 — BASE DE DONNÉES : NOTIONS DE BASE

1.1 Qu'est-ce qu'une base de données ?

Une base de données est :

Un système qui permet de stocker des informations de manière organisée.

Exemple (agence immobilière) :

Table users → utilisateurs
Table properties → biens immobiliers
Table clients → clients

1.2 Qu'est-ce que MySQL ?

MySQL est :

Un système de gestion de base de données relationnelle.

Laravel communique avec MySQL pour :

- Lire des données
- Ajouter des données
- Modifier
- Supprimer

PARTIE 2 — CONFIGURATION DE LA BASE DE DONNÉES

Où configurer la base ?

Dans deux fichiers :

1/ .env
2/ config/database.php

Le plus important pour débutants : .env

Fichier .env

Chemin :

/.env

Ouvrir le fichier et chercher :

DB_CONNECTION=

DB_HOST=

DB_PORT=

DB_DATABASE=

DB_USERNAME=

DB_PASSWORD=

Exemple configuration MySQL

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=agence

DB_USERNAME=root

DB_PASSWORD=

Explication :

DB_CONNECTION=mysql

Indique à Laravel :

- Utilise MySQL comme base de données.

DB_HOST=127.0.0.1

Adresse du serveur.

127.0.0.1 = localhost = votre ordinateur.

DB_PORT=3306

Port utilisé par MySQL.

3306 est le port par défaut.

DB_DATABASE=agence

Nom de la base.

⚠ IMPORTANT
La base "agence" doit déjà exister dans phpMyAdmin.

DB_USERNAME=root

Nom utilisateur MySQL.

DB_PASSWORD=

Mot de passe MySQL.

PARTIE 3 — TESTER LA CONNEXION

Étape 1 : Ouvrir le terminal

Dans le dossier du projet Laravel.

Étape 2 : Exécuter

php artisan migrate

Que fait cette commande ?

1. Laravel lit le fichier .env
2. Tente de se connecter à MySQL
3. Crée les tables système

Si erreur :

Exemple :

Unknown database 'agence'

→ La base n'existe pas.

Solution **: créer la base dans phpMyAdmin.**

PARTIE 4 — UTILISER LA FACADE DB

Qu'est-ce que la façade DB ?

Classe Laravel :

Illuminate**Support\Facades\DB**

Elle permet :

- **Exécuter des requêtes SQL**
- **Gérer transactions**
- **Manipuler données**

Où l'utiliser ?

Dans :

routes/web.php

Ajouter l'import

En haut du fichier :

use Illuminate**Support\Facades\DB;**

Exemple test connexion

Dans routes/web.php :

```
Route::get('/test-db', function () {
    $result = DB::select('SELECT 1');
    return "Connexion réussie";
});
```

Explication

Route::get()

Crée une route.

DB::select()

Exécute une requête SELECT.

'SELECT 1'

Petite requête simple pour tester.

PARTIE 5 — CRUD EN SQL

Définition CRUD

Lettre	Signification
C	**Create**
R	**Read**
U	**Update**
D	**Delete**

CREATE — Insérer

```
DB::insert(
   'INSERT INTO users (name, email, password) VALUES (?, ?, ?)',
   ['Ali', 'ali@mail.com', bcrypt('123456')]
);
```

Explication :

DB::insert()

Méthode pour insérer des données.

?

Paramètres sécurisés.

⚠ Évite l'injection SQL.

bcrypt('123456')

Fonction Laravel qui chiffre le mot de passe.

READ — Lire

```
$users = DB::select('SELECT * FROM users');
```

Retourne :

→ **Tableau d'objets.**

Affichage

```
foreach ($users as $user) {
   echo $user->name;
}
```

UPDATE — Modifier

```
DB::update(
   'UPDATE users SET name = ? WHERE id = ?',
   ['Karim', 1]
);
```

DELETE — Supprimer

```
DB::delete(
   'DELETE FROM users WHERE id = ?',
   [1]
);
```

PARTIE 6 — TRANSACTIONS

Définition

Une transaction est :

Un groupe d'opérations exécutées comme un bloc unique.

Si une échoue → tout est annulé.

Exemple

```
DB::beginTransaction();
try {
   DB::insert(
      'INSERT INTO users (name) VALUES (?)',
      ['Test']
   );
   DB::commit();
   return "Succès";
} catch (\Exception $e) {
   DB::rollBack();
   return "Erreur";
}
```

Explication

beginTransaction()

Démarre transaction.

commit()

Valide définitivement.

rollBack()

Annule tout

PARTIE 7 — MIGRATIONS

Définition

Une migration est :

Un **fichier PHP** qui **crée** ou **modifie** une table.

Créer migration

php artisan make:**migration create_properties_table**

Fichier créé dans :

database/migrations/

Exemple

```
public function up()
{
   Schema::create('properties', function (Blueprint $table) {
      $table->id();
      $table->string('title');
      $table->integer('price');
      $table->timestamps();
   });
}
```

Explication

Schema::create()

Crée table.

$table->id()

Clé primaire auto-incrémentée.

string()

Champ texte.

integer()

Nombre entier.

timestamps()

created_at et updated_at.

Exécuter migration

php artisan **migrate**

PARTIE 8 — SEEDERS

Définition

Un **seeder** est :

Un **fichier** qui **insère** automatiquement des **données**.

Pourquoi on l'utilise ?

Quand on développe une application, on a besoin :

- De données pour tester
- D'un compte admin par défaut
- De données de démonstration
- De remplir rapidement la base

Au lieu d'insérer les données manuellement dans phpMyAdmin,
on utilise un **Seeder**.

Étape 1 : Créer seeder

php artisan **make:seeder UserSeeder**

Fichier créé :

database/seeders/**UserSeeder.php**

Étape 2 : Modifier le fichier

```
use Illuminate\Support\Facades\DB;

public function run()

{

    DB::insert(

        'INSERT INTO users (name, email, password) VALUES (?, ?, ?)',

        ['Admin', 'admin@mail.com', bcrypt('123456')]

    );

}
```

Étape 3 : Déclarer le seeder

Ouvrir :

database/seeders/**DatabaseSeeder.php**

Ajouter :

$this->call(UserSeeder::class);

Étape 4 : Exécuter

php artisan db:seed

PARTIE 9 — CLI (Command Line Interface)

Définition

CLI = interface ligne de commande (terminal).

Commandes importantes

Commande	Rôle
php artisan migrate	Créer tables
php artisan migrate:rollback	Annuler
php artisan migrate:fresh	Supprimer tout
php artisan make:migration	Créer migration
php artisan make:seeder	Créer seeder
php artisan db:seed	Exécuter seeder

TP GUIDÉ – Base de Données Laravel

Thème : Mini Système Agence Immobilière

Dans ce TP, vous allez :

- **Configurer une base de données**
- **Créer une table avec migration**
- **Insérer des données**
- **Lire des données**
- **Modifier des données**
- **Supprimer des données**
- **Utiliser une transaction**
- **Créer des seeders**
- **Manipuler Artisan (CLI)**

Vous devez suivre les étapes dans l'ordre.

PARTIE 1 — CONFIGURATION

Étape 1 : Créer la base de données

Ouvrir **phpMyAdmin**

Créer une base appelée :

agence_tp

Étape 2 : Configurer Laravel

Ouvrir le fichier :

.env

Modifier :

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=agence_tp

DB_USERNAME=root

DB_PASSWORD=

Enregistrer le fichier.

Étape 3 : Tester la connexion

Dans le terminal (dans le dossier du projet) :

php artisan migrate

Si aucune erreur → connexion OK.

PARTIE 2 — CRÉER LA TABLE PROPERTIES

Étape 4 : Créer une migration

Dans le terminal :

php artisan make:migration create_properties_table

Étape 5 : Modifier la migration

Ouvrir :

database/migrations/xxxx_create_properties_table.php

Dans la méthode up() écrire :

```
public function up()
{
    Schema::create('properties', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->integer('price');
        $table->string('city');
        $table->timestamps();
    });
}
```

Étape 6 : Exécuter la migration

php artisan migrate

Vérifier dans phpMyAdmin :
La table properties doit exister.

PARTIE 3 — INSÉRER DES DONNÉES

Étape 7 : Ouvrir routes/web.php

Ajouter en haut :

```
use Illuminate\Support\Facades\DB;
```

Étape 8 : Ajouter une route d'insertion

```
Route::get('/insert-property', function () {

    DB::insert(
        'INSERT INTO properties (title, price, city) VALUES (?, ?, ?)',
        ['Appartement centre', 750000, 'Casablanca']
    );

    return "Bien ajouté avec succès";

});
```

Étape 9 : Tester

Lancer le serveur :

```
php artisan serve
```

Dans le navigateur :

http://127.0.0.1:8000/insert-property

Vérifier dans phpMyAdmin → une ligne ajoutée.

PARTIE 4 — AJOUTER 3 BIENS DIFFÉRENTS

Étape 10 : Ajouter une nouvelle route

Toujours dans routes/web.php :

```
Route::get('/insert-multiple', function () {
```

```
    DB::insert(
        'INSERT INTO properties (title, price, city) VALUES (?, ?, ?)',
        ['Villa luxe', 2500000, 'Rabat']
    );
    DB::insert(
        'INSERT INTO properties (title, price, city) VALUES (?, ?, ?)',
        ['Studio moderne', 450000, 'Casablanca']
    );
    DB::insert(
        'INSERT INTO properties (title, price, city) VALUES (?, ?, ?)',
        ['Maison familiale', 1200000, 'Marrakech']
    );
    return "3 biens ajoutés";
});
```

Étape 11 : Tester

Aller à :

/insert-multiple

Vérifier dans la base :
Vous devez avoir maintenant 4 biens.

PARTIE 5 — AFFICHER TOUS LES BIENS

Étape 12 : Ajouter route affichage

```
Route::get('/properties', function () {
    $properties = DB::select('SELECT * FROM properties');
    foreach ($properties as $property) {
        echo $property->title . " - " . $property->price . " - " . $property->city . "<br>";
    }
});
```

Étape 13 : Tester

Aller à :

/properties

Tous les biens doivent s'afficher.

PARTIE 6 — AFFICHER UNIQUEMENT CASABLANCA

Étape 14 : Ajouter route filtrée

```
Route::get('/properties-casa', function () {
    $properties = DB::select(
        'SELECT * FROM properties WHERE city = ?',
        ['Casablanca']
    );
    foreach ($properties as $property) {
        echo $property->title . " - " . $property->price . "<br>";
    }
});
```

Étape 15 : Tester

Aller à :

/properties-casa

Seuls les biens de Casablanca doivent apparaître.

PARTIE 7 — MODIFIER UN BIEN

Étape 16 : Ajouter route update

```
Route::get('/update-property', function () {
    DB::update(
        'UPDATE properties SET price = ? WHERE id = ?',
        [900000, 1]
    );
    return "Prix modifié";
```

```
});
```

Étape 17 : Tester

Aller à :

/update-property

Puis vérifier dans la base si le prix du bien ID 1 a changé.

PARTIE 8 — SUPPRIMER UN BIEN

Étape 18 : Ajouter route delete

```
Route::get('/delete-property', function () {
    DB::delete(
        'DELETE FROM properties WHERE id = ?',
        [2]
    );
    return "Bien supprimé";

});
```

Étape 19 : Tester

Aller à :

/delete-property

Vérifier dans phpMyAdmin :
Le bien ID 2 doit être supprimé.

PARTIE 9 — TRANSACTION

Étape 20 : Ajouter route transaction

```
Route::get('/transaction-test', function () {
    DB::beginTransaction();
```

```
    try {
        DB::insert(
            'INSERT INTO properties (title, price, city) VALUES (?, ?, ?)',

            ['Projet transaction 1', 1000000, 'Agadir']
        );
        DB::insert(
            'INSERT INTO properties (title, price, city) VALUES (?, ?, ?)',
            ['Projet transaction 2', 1100000, 'Agadir']
        );
        DB::commit();
        return "Transaction réussie";
    } catch (\Exception $e) {
        DB::rollBack();

        return "Erreur, tout annulé";
    }
});
```

Tester avec :

/transaction-test

PARTIE 10 — SEEDER 1

Étape 21 : Créer seeder

php artisan make:seeder PropertySeeder

Étape 22 : Modifier fichier

Ouvrir :

database/seeders/PropertySeeder.php

Écrire :

```
use Illuminate\Support\Facades\DB;
```

```
public function run()
{
    DB::insert(
        'INSERT INTO properties (title, price, city) VALUES (?, ?, ?)',
        ['Studio Seeder', 300000, 'Fès']
    );
}
```

Étape 23 : Déclarer dans DatabaseSeeder

Ouvrir :

database/seeders/DatabaseSeeder.php

Ajouter :

```
$this->call(PropertySeeder::class);
```

Étape 24 : Exécuter

```
php artisan db:seed
```

PARTIE 11 — DEUXIÈME SEEDER (2 BIENS)

Étape 25 : Créer un nouveau seeder

```
php artisan make:seeder PropertySeederTwo
```

Étape 26 : Modifier le fichier

```
use Illuminate\Support\Facades\DB;

public function run()
{
    DB::insert(
        'INSERT INTO properties (title, price, city) VALUES (?, ?, ?)',
```

```
['Appartement vue mer', 2000000, 'Tanger']
    );

    DB::insert(
        'INSERT INTO properties (title, price, city) VALUES (?, ?, ?)',
        ['Terrain agricole', 600000, 'Meknès']
    );
}
```

Étape 27 : Déclarer dans DatabaseSeeder

Ajouter :

```
$this->call(PropertySeederTwo::class);
```

Étape 28 : Exécuter

```
php artisan db:seed
```